OTA Device Emulation

Field	Value
Revision	1
Last modified	2026-06-10T00:00:00Z
Status	active
Scope	Tier-1 OTA control-plane testing via a podman/ docker device-emulator container

This document describes the `ota-device-emu` container image (`images/ota-device-emu/`) and the on-demand boot/health recipe (`cmd/ota-device-emu-boot/`) added to the `digital.vasic.containers` module so a consuming project can run an **emulated OTA â€œdeviceâ€ï** as a container in place of live hardware for control-plane testing.

The image and recipe are **project-agnostic** (Constitution CONST-051(B)): every project-specific value (the binary, the control-plane image, the OTA_* configuration) is supplied by the consumer at run time. Nothing about any particular consumer is baked into the module.

What this is â€” and what it is NOT (honest tiered boundary)

The OTA update system has two layers, and this image addresses only the first.

Layer	What it exercises	This image?
Tier-1: control-plane protocol	enrollment, update-check, version reporting, campaign/ rollout assignment, heartbeat â€” the HTTP request/response contract between a device and the OTA server	**YES** â€” a Linux container running a Go client binary that speaks the protocol
Tier-2: on-device A/B apply	real Android A/B `update_engine` + AVB/dm-verity slot flip + auto-rollback on the RK3588 / Orange Pi 5 Max target	**NO**

Why Tier-2 is out of scope here (FACT, not guess). The real Android A/B flow needs `update_engine`, AVB/dm-verity, and a bootloader slot mechanism running under an Android system image. On this host that means **Cuttlefish on Linux with KVM** (`/dev/kvm`) â€” see the existing `pkg/emulator` notes on the Linux-x86_64-KVM requirement. The macOS/applehv development host used to build and validate this image has **no KVM and no Android `emulator`**, so the runnable mechanism here is a **podman container**, not a host-AVD/Cuttlefish device. Driving the genuine `update_engine` apply belongs to a Cuttlefish-on-Linux-KVM image (a separate, future deliverable), NOT to this control-plane emulator.

A consumer that needs the real apply flow runs the Tier-2 path on a Linux+KVM host (or real hardware); this Tier-1 image lets the control-plane server be developed and tested continuously, on any host, without that hardware.

The contract: who builds what

```
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
â”‚ consuming project          â”‚       â”‚ digital.vasic.containers (t
â”‚ (an OTA control-plane consumer)    â”‚       â”‚ submodule)
â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
â”‚ builds the static Go binary â”‚ â”€â”€â”€â”€â–¶ â”‚ images/ota-devi
â”‚   ota-device-emu           â”‚ binary â”‚   provides RUNTIME + entryp
â”‚ (CGO_ENABLED=0, linux)     â”‚        â”‚ cmd/ota-device-emu-boot
â”‚ supplies it via volume     â”‚        â”‚   provides pkg/boot + pkg/h
â”‚ mount or a derived image   â”‚        â”‚   on-demand bring-up recipe
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
```

- **The submodule provides the runtime, not the binary.** The image is a minimal Alpine base + CA certs + a POSIX entrypoint. It does **not** build `ota-device-emu` â€” that binary lives in (and is built by) the consuming project, which knows the OTA protocol.
- **Two supply paths**, both validated:

  1. **Volume mount** (dev default): `-v ./bin/ota-device-emu:/usr/local/bin/ota-device-emu:ro`

  2. **Derived image** (CI / pinned):

     ```
     FROM ota-device-emu:dev
     COPY bin/ota-device-emu /usr/local/bin/ota-device-emu
     ```

Env-var interface

The entrypoint reads these and hands them to the binary (the binary itself consumes them â€” the entrypoint only validates the required one and prints the contract on `--help`).

Variable	Required	Meaning
`OTA_BASE_URL`	**yes**	Control-plane base URL, e.g.Â `http://control-plane:8080`
`OTA_ADMIN_USER`	no	Enrollment/admin user (if the binary authenticates)
`OTA_ADMIN_PASS`	no	Enrollment/admin password â€” **never logged** (Constitution Â§11.4.10)
`OTA_HARDWARE_ID`	no	Stable hardware id this emulated device reports
`OTA_MODEL`	no	Device model, e.g.Â `orangepi-5-max`
`OTA_OS`	no	Device OS, e.g.Â `android` / `linux`

Variable	Required	Meaning
`OTA_CURRENT_VERSION`	no	Version the emulated device currently runs
`OTA_DEVICE_EMU_BIN`	no	Binary path (default `/usr/local/bin/ota-device-emu`)
`OTA_DEVICE_EMU_EXTRA_ARGS`	no	Extra args appended to the binary invocation

Entrypoint exit codes (honest, no silent success):

Exit	Condition
`0`	`--help`/`-h` (prints the contract), or the binary ran and exited 0
`2`	`OTA_BASE_URL` unset when a binary is present
`3`	binary absent/not-executable at `OTA_DEVICE_EMU_BIN`

On-demand-infra invariant (Constitution Â§11.4.76)

Operators must **never** be required to run `podman compose up` by hand. The boot is part of the test entry point: `cmd/ota-device-emu-boot` brings the stack up via `pkg/boot.BootManager` (which runs the compose `Up`) and health-checks it via `pkg/health`. It composes with â€” and does not reimplement â€” the existing `pkg/boot` + `pkg/compose` + `pkg/health` surfaces (Â§11.4.74 extend-donâ€™t-reimplement).

How it composes with a control-plane container

The example compose file (`images/ota-device-emu/docker-compose.example.yml`) declares two services:

- `control-plane` â€” the OTA server under test, exposing an HTTP health endpoint (e.g.Â `/health` on `:8080`);
- `ota-device-emu` â€” this image, with `OTA_BASE_URL` pointing at `http://control-plane:8080` and the device-identity env set.

`ota-device-emu-boot` then asserts **positive evidence two ways** (anti-bluff, Â§11.4 / Â§11.4.69):

1. the **control plane answers its HTTP health endpoint** (`pkg/health` HTTP checker) â€” proof the server the device talks to is actually up;
2. the **device-emulator container is genuinely RUNNING**, queried from the container runtime (`ContainerRuntime.Status`), **not** a fake port probe. The emulator is an *outbound* client (it polls/heartbeats the control plane) and has no inbound port, so its real liveness oracle is â€œthe container the runtime reports is in state `running`â€, not a synthetic TCP check. Asserting a port that doesnâ€™t exist would be exactly the metadata-only PASS-bluff Â§11.4 forbids.

A consumer wanting a stronger end-to-end proof (the device actually appeared in the control planeâ€™s registry) adds a control-plane-side query â€” e.g.Â an HTTP check of `/v1/devices/<OTA_HARDWARE_ID>` â€” as a third `pkg/health` HTTP target. That is the

genuine sink-side evidence (Â§11.4.13) and is the recommended next step once the control-plane image is wired.

Quick start

```
# 0. (consumer) build the static device-emulator binary in the OTA control
#    CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build -o bin/ota-device-emu

# 1. build the runtime image (this submodule):
podman build -f images/ota-device-emu/Dockerfile \
  -t ota-device-emu:dev images/ota-device-emu/

# 2. inspect the contract:
podman run --rm ota-device-emu:dev --help

# 3. bring the stack up on-demand + health-check it (no manual compose up)
go run ./cmd/ota-device-emu-boot \
  --compose images/ota-device-emu/docker-compose.example.yml \
  --control-port 8080 --control-health-path /health
```

Host requirements

- **podman** (or docker) with a running machine/daemon. On macOS/applehv the podman machine provides a Linux arm64 VM; the image is multi-arch (Alpine).
- **No KVM / no Android** `emulator` **needed** â€” that is the whole point of the Tier-1 container path. The Tier-2 Cuttlefish path (future) does need Linux + `/dev/kvm`.

Anti-bluff posture (Constitution Â§11.4)

- The imageâ€™s `--help`, missing-binary (exit 3), and missing-`OTA_BASE_URL` (exit 2) paths were each run under real podman and produce honest, distinguishable results â€” never a silent PASS.
- The boot recipe asserts the **real** observable for each service (HTTP health for the control plane; runtime `running` state for the emulator), not a config-only or absence-of-error check.
- This image addresses Tier-1 only; the Tier-2 boundary above is stated as fact with its cause (no KVM / no host emulator), per the no-guessing mandate (Â§11.4.6).

Files

Path	Purpose
`images/ota-device-emu/Dockerfile`	Runtime image (Alpine + CA certs + entrypoint)
`images/ota-device-emu/entrypoint.sh`	Env validation + contract + binary hand-off
`images/ota-device-emu/docker-compose.example.yml`	Example control-plane + emulator stack
`cmd/ota-device-emu-boot/main.go`	On-demand boot + health recipe (pkg/boot + pkg/health)
`docs/ota-device-emulation.md`	This document

Sibling exports (Constitution Â§11.4.65)

`.html` and `.pdf` siblings of this document are generated via pandoc + weasyprint and committed alongside the markdown. This submodule has no in-repo `sync_all_markdown_exports.sh`; the parent projectâ€™s export tooling keeps the wider doc fleet in sync. To regenerate locally:

```
pandoc docs/ota-device-emulation.md -o docs/ota-device-emulation.html
weasyprint docs/ota-device-emulation.html docs/ota-device-emulation.pdf
```